

LAB 4 – VIBE CODING

Chat History & Implementation Record

Project: Pygame Frogger

Repository: SETAPESU26/01_frogger

Student Fork: abdulmuheethk-1026/02_frogger

Environment: Windows 11 · Python 3.13.15 · Pygame 2.6.1

This document records the Vibe Coding workflow used to modify the assigned Frogger project. The four requested tasks were implemented sequentially and committed separately.

1. Task 1 – Fix Vehicle Collision Detection

Objective

Fix the broken vehicle collision detection so that a collision is detected using the actual positions and dimensions of the frog and vehicles.

Vibe Coding instruction

Continue the Vibe Coding lab on the existing Pygame Frogger project. Implement Task 1 only: fix vehicle collision detection. Use the actual pygame rectangles of the frog and vehicles so collision is detected correctly.

Implementation recorded

The collision function was updated to obtain the frog rectangle and each vehicle rectangle using `get_rect(CELL_SIZE)`, then test them with `pygame.Rect.colliderect()`.

File changed

`game/collisions.py`

Testing / verification

The updated game was run and the collision behavior was checked before proceeding to Task 2.

Git commit

347f78d – Task 1: Fix vehicle collision detection

2. Task 2 – Three Lives, Respawn and Game Over

Objective

Add three lives, respawn the frog after a vehicle collision, and display Game Over when all lives are lost.

Vibe Coding instruction

Yes, let's move to Task 2. Add three lives with respawn after vehicle collision and a Game Over state. Build this on the existing Task 1 implementation.

Implementation recorded

game/game_engine.py was extended with MAX_LIVES = 3, a lives counter, and a game_over state. A collision decreases the number of lives. If lives remain, the frog is respawned; when the final life is lost, the game enters Game Over. Reset functionality restores the initial state.

Additional behavior

Movement is blocked while Game Over is active, and the R key can reset the game.

Testing / verification

The three-life behavior, respawn behavior, and Game Over state were tested before moving to Task 3.

Git commit

17f7f01 – Task 2: Add lives and game over

3. Task 3 – Goal Detection, Scoring and Win State

Objective

Detect when the frog reaches the goal, increase the score, and display a win state after the required number of goals.

Vibe Coding instruction

Yes, let's continue with the next task. Add goal detection, scoring, and a Win state to the existing Frogger implementation.

Implementation recorded

game/game_engine.py was extended with GOALS_TO_WIN = 3, a score counter, and a won state. When the frog reaches GOAL_ROW, the score is incremented and the frog is reset. When the score reaches 3, the game enters the Win state.

Display changes

The game displays Score: x/3 and shows YOU WIN! after the required number of goals.

Testing / verification

Goal detection, score incrementing, frog reset after a goal, and the final Win state were tested.

Git commit

d7b3fe7 – Task 3: Add goal scoring and win state

4. Task 4 – 30-Second Countdown Timer and Timeout

Objective

Add a real-time 30-second countdown and end the game with a timeout when the timer reaches zero.

Vibe Coding instruction

I am continuing the Vibe Coding lab on the existing Pygame Frogger project. Tasks 1, 2, and 3 have already been implemented and tested. Now implement ONLY Task 4: Add a 30-second countdown timer and timeout handling.

Implementation requirements recorded

Start a 30-second timer when the game begins. Update the remaining time using real elapsed time rather than assuming a fixed number of frames. When the timer reaches zero, stop normal gameplay, set the game to Game Over, and distinguish the timeout condition from a collision-based Game Over.

Implementation recorded

game/game_engine.py was extended with GAME_DURATION = 30.0, timeout state, time_remaining, and last_time. pygame.time.get_ticks() is used to calculate elapsed time between updates. The remaining time is clamped to zero. At zero, game_over is set and timeout is marked true. Reset restores the timer to 30 seconds.

Display changes

The HUD displays the remaining time. The game distinguishes YOU WIN!, TIME OUT!, and GAME OVER! states.

Testing / verification

The timer countdown and timeout behavior were tested before finalizing the fourth task.

Git commit

7fd71ac – Task 4: Add countdown timer and timeout

5. Git Workflow and Final Submission

Separate commits

Task	Commit	Description
Task 1	347f78d	Fix vehicle collision detection
Task 2	17f7f01	Add lives and game over
Task 3	d7b3fe7	Add goal scoring and win state
Task 4	7fd71ac	Add countdown timer and timeout

Repository workflow

The initial attempt to push directly to the SETAPESU26 repository was rejected because the account did not have write permission.

The project was then prepared for the student's own fork. The final fork name is 02_frogger under the student's GitHub account.

Final repository remote

```
https://github.com/abdulmuheethk-1026/02_frogger
```

Final working-tree status

The local repository was clean after the four task commits, with no uncommitted changes.

Submission checklist

Deliverable	Status / file
Before video	10-second baseline recording
After video	10-second recording after all changes
Updated code	Frogger project with Tasks 1–4
Chat history	This PDF
GitHub	Student fork: 02_frogger

Note: This is a lab-focused record of the Vibe Coding work and implementation decisions. It is intended as a clean submission document rather than a raw browser printout of the entire ChatGPT interface.